

38. Build an end-to-end RAG pipeline involving document loading, text chunking, embeddings, retrieval, and answer generation.

Code:

```
from sentence_transformers import SentenceTransformer
from transformers import pipeline
import faiss
import numpy as np
```

```
# 1. Document loading
```

```
print("Loading document...")
```

```
document = """
```

```
Artificial Intelligence is a branch of computer science.
```

```
It focuses on creating systems that can perform tasks
that normally require human intelligence.
```

```
Machine Learning is a subset of Artificial Intelligence.
```

```
It enables computers to learn patterns from data and
make predictions without being explicitly programmed.
```

```
Deep Learning is a subset of Machine Learning.
```

```
It uses artificial neural networks with multiple layers
to learn complex patterns from large amounts of data.
```

```
Natural Language Processing is a branch of Artificial
Intelligence that enables computers to understand,
process and generate human language.
```

Computer Vision enables computers to understand and analyze images and videos.

```
"""
```

```
print("Document loaded successfully.")
```

```
# 2. Text chunking
```

```
print("\nCreating text chunks...")
```

```
chunks = [
```

```
    chunk.strip()
```

```
    for chunk in document.split("\n\n")
```

```
    if chunk.strip()
```

```
]
```

```
print("Number of chunks:", len(chunks))
```

```
# 3. Load embedding model
```

```
print("\nLoading embedding model...")
```

```
embedding_model = SentenceTransformer(
```

```
    "all-MiniLM-L6-v2"
```

```
)
```

```
# 4. Generate embeddings
```

```
print("Creating embeddings...")
```

```
embeddings = embedding_model.encode(chunks)
```

```
embeddings = np.array(embeddings).astype("float32")
```

```
# 5. Create FAISS vector database
```

```
print("Creating FAISS vector database...")
```

```
dimension = embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(embeddings)
```

```
print("Documents stored successfully.")
```

```
print("Total vectors:", index.ntotal)
```

```
# 6. Load language model
```

```
print("\nLoading language model...")
```

```
qa_model = pipeline(  
    "text2text-generation",  
    model="google/flan-t5-small"  
)
```

```
print("Language model loaded successfully.")
```

7. Get user question

```
question = input("\nEnter your question: ")
```

8. Create query embedding

```
query_embedding = embedding_model.encode(  
    [question]  
)
```

```
query_embedding = np.array(  
    query_embedding  
).astype("float32")
```

9. Retrieve relevant chunks

```
print("\nSearching for relevant information...")
```

```
k = 2
```

```
distances, indices = index.search(  
    query_embedding,  
    k  
)
```

```
retrieved_chunks = []
```

```
for i in indices[0]:
```

```
    retrieved_chunks.append(chunks[i])
```

```
context = "\n".join(retrieved_chunks)
```

```
# 10. Generate answer
```

```
prompt = """
```

```
Answer the question using only the information  
provided in the context.
```

```
Context:
```

```
{}
```

```
Question:
```

```
{}
```

```
Answer:
```

```
""".format(context, question)
```

```
print("Generating answer...")
```

```
result = qa_model(
```

```
    prompt,
```

```
    max_new_tokens=100
```

```
)
```

```
answer = result[0]["generated_text"]
```

```
# 11. Display result
```

```
print("\n=====")
```

```
print("    RAG FINAL RESULT")
```

```
print("=====")
```

```
print("\nQuestion:")
```

```
print(question)
```

```
print("\nRetrieved Context:")
```

```
print(context)
```

```
print("\nGenerated Answer:")
```

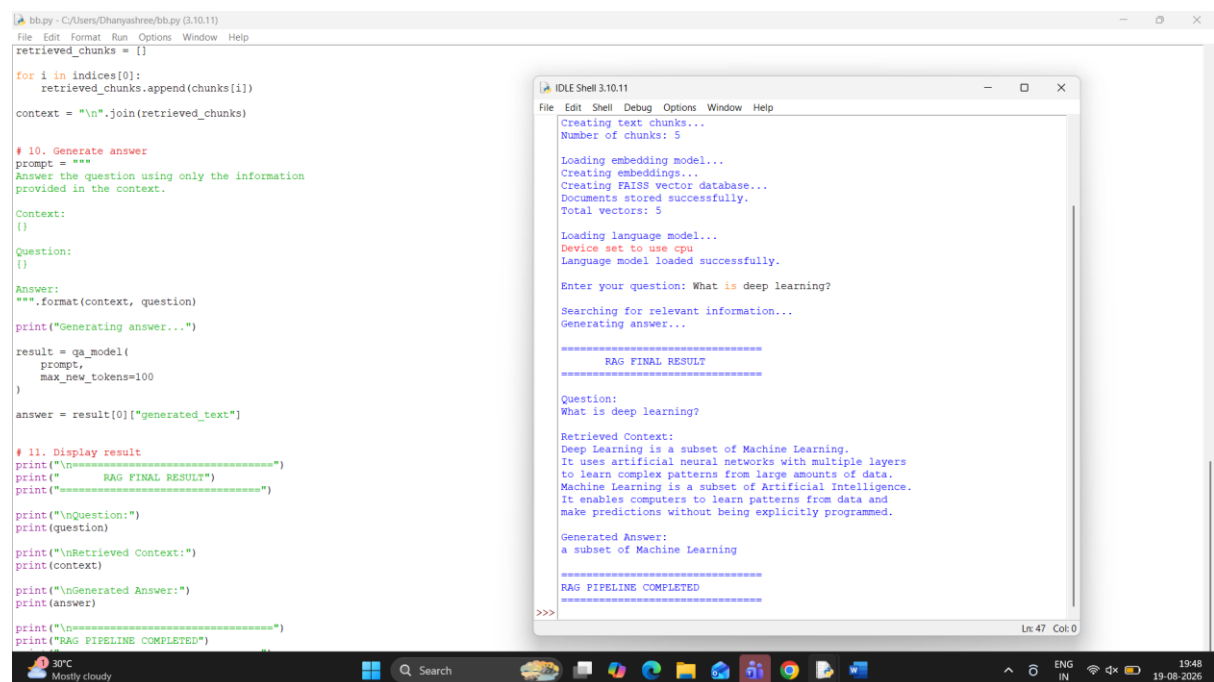
```
print(answer)
```

```
print("\n=====")
```

```
print("RAG PIPELINE COMPLETED")
```

```
print("=====")
```

output:



The image shows a screenshot of a Python script in a file editor and its execution output in an IDLE Shell window. The script implements a Retrieval-Augmented Generation (RAG) pipeline. It starts by creating text chunks from a document, embedding them, and storing them in a FAISS vector database. Then, it loads a language model and uses it to generate an answer based on a user question and the retrieved context. The output window shows the progress of these steps, the user's question, the retrieved context, and the generated answer.

```
bb.py - C:/Users/Dhanyashree/bb.py (3.10.11)
File Edit Format Run Options Window Help

retrieved_chunks = []

for i in indices[0]:
    retrieved_chunks.append(chunks[i])

context = "\n".join(retrieved_chunks)

# 10. Generate answer
prompt = """
Answer the question using only the information
provided in the context.

Context:
{}

Question:
{}

Answer:
""".format(context, question)
print("Generating answer...")

result = qa_model(
    prompt,
    max_new_tokens=100
)

answer = result[0]["generated_text"]

# 11. Display result
print("\n=====")
print("    RAG FINAL RESULT")
print("=====")

print("\nQuestion:")
print(question)

print("\nRetrieved Context:")
print(context)

print("\nGenerated Answer:")
print(answer)

print("\n=====")
print("RAG PIPELINE COMPLETED")
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Creating text chunks...
Number of chunks: 5

Loading embedding model...
Creating embeddings...
Creating FAISS vector database...
Documents stored successfully.
Total vectors: 5

Loading language model...
Device set to use cpu
Language model loaded successfully.

Enter your question: What is deep learning?

Searching for relevant information...
Generating answer...

=====
RAG FINAL RESULT
=====

Question:
What is deep learning?

Retrieved Context:
Deep Learning is a subset of Machine Learning.
It uses artificial neural networks with multiple layers
to learn complex patterns from large amounts of data.
Machine Learning is a subset of Artificial Intelligence.
It enables computers to learn patterns from data and
make predictions without being explicitly programmed.

Generated Answer:
a subset of Machine Learning

=====
RAG PIPELINE COMPLETED
=====
>>>
```

30°C Mostly cloudy 19:48 19-08-2026